

## Frequency Analysis and Synthesis

A conversion from time-domain samples to frequency-domain coefficients is the first stage in all of the current standard digital audio compression algorithms. In the MPEG suite of algorithms, this conversion is done using an efficient implementation of a subband filterbank approach, whereas in the Dolby AC-3 algorithm, a cosine transform is used that incorporates time-domain aliasing cancellation. In this chapter, the basic theory behind each of these techniques is explained and the actual implementations used in the standard algorithms are discussed.

### 8.1. THE SAMPLING THEOREM

The first stage in digital audio coding is usually the conversion of the analog waveform into a series of digital samples. Probably the most important concept to consider during this conversion is the introduction of *aliasing* errors. In this section, the mathematical explanation for aliasing, which is more commonly known as *the sampling theorem*, is introduced.

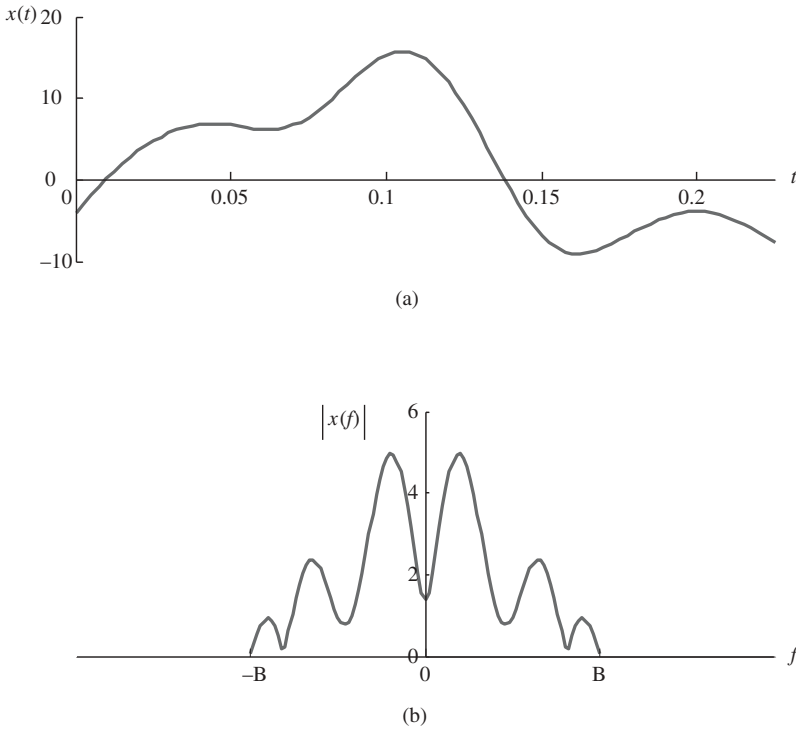
Consider the signal  $x(t)$  that has a spectrum  $X(f)$  and a bandwidth of  $B$  Hz as shown in Figure 8.1(a) and (b).

If this signal is sampled at a rate of one sample every  $T_s$  seconds, this is equivalent to multiplying the signal by a unit impulse train with period  $T_s$ . The sampled signal  $x(nT_s)$  is given by the following equation:

$$x(nT_s) = x(t) \sum_{n=-\infty}^{\infty} \delta(t - nT_s) \quad (8.1)$$

By taking the exponential Fourier series of an impulse train, this equation can be rewritten to give

$$x(nT_s) = \frac{1}{T_s} \sum_{n=-\infty}^{\infty} x(t) e^{jn\omega_s t} \quad (8.2)$$



**Figure 8.1** (a) The signal  $x(t)$  and (b) its magnitude spectrum  $|X(f)|$ .

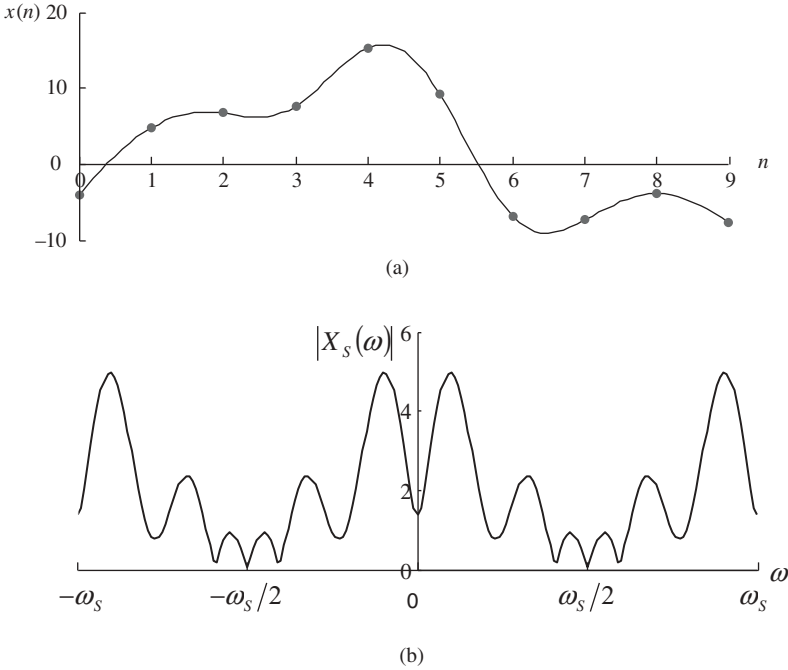
and taking the Fourier transform of  $x(nT_s)$  gives

$$X_s(\omega) = \frac{1}{T_s} \sum_{n=-\infty}^{\infty} X(\omega - n\omega_s) \quad (8.3)$$

The spectrum of the sampled signal therefore consists of the spectrum of the original signal repeating itself indefinitely at intervals of  $\omega_s = 2\pi f_s$ , where  $f_s = 1/T_s$  is the *sampling frequency*. Figure 8.2(a) shows the sampled signal (in future, for ease of notation, the sampling period term  $T_s$  will be dropped and this signal will simply be referred to as  $x(n)$ ), and Figure 8.2(b) shows the magnitude spectrum of the sampled signal.

Figure 8.2(b) shows that there is no overlap between successive cycles of  $X_s(\omega)$  provided that the sampling frequency  $\omega_s$  is greater than twice the maximum frequency of the signal. If there is no overlap between successive cycles, the original signal can be recovered from the sampled signal by passing it through a low-pass filter. The sampling theorem states that a signal with a bandwidth of  $B$  Hz can be reconstructed perfectly from samples taken at a rate of not less than  $2B$  samples per second. This sampling rate of  $2B$  samples per second is known as the *Nyquist sampling rate*.

If a signal is sampled at a rate less than the Nyquist rate, the spectrum of the sampled signal consists of overlapping repetitions of  $X(\omega)$ . If the sampled signal



**Figure 8.2** (a) The sampled signal  $x(n)$  and (b) its magnitude spectrum  $|X_s(\omega)|$ .

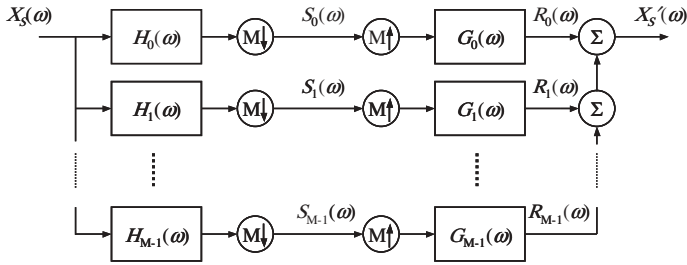
is then passed through a low-pass filter, the result is a distorted version of the original signal. This distortion is known as *aliasing error* and is caused by the loss of frequency components above the frequency of  $\omega_s/2$  and frequency components from the next repetition of  $X(\omega)$  adding to the original signal below the frequency of  $\omega_s/2$ .

## 8.2. DIGITAL FILTERS

For the MPEG suite of audio coding algorithms, the conversion from time-domain samples to frequency coefficients is performed using a bank of *digital filters*. A digital filter is specified by its discrete impulse response  $h(k)$ ,  $k = 0, \dots, N$ . A sampled signal  $x(n)$  is *filtered* by performing a discrete convolution of the filter impulse response with the sampled signal. Hence the output of a digital filter with impulse response  $h(k)$  is given by the equation

$$s(n) = \sum_{k=0}^N h(k) x(n-k) \quad (8.4)$$

The frequency response of a digital filter can be found by taking the Fourier transform of the filter impulse response.



**Figure 8.3**  $M$ -channel subband analysis and synthesis filters.

### 8.3. SUBBAND FILTERING

Subband filtering is the process of dividing the original signal into a series of frequency bands. This results in a set of frequency coefficients or *subband samples* that represent the amplitude of the component with the frequency of the corresponding subband. These subband samples are then transmitted and added together to reconstruct the original signal. The subband filtering process requires two steps: First, the analysis filter bank is used to convert time-domain samples of the original signal to frequency-domain subband samples. The synthesis filter bank is then used to reconstruct the time-domain samples of the signal from the transmitted subband samples. The block diagram of an  $M$ -channel analysis–synthesis filter bank is shown in Figure 8.3.

#### 8.3.1. The Analysis Filter Bank

The analysis stage uses a set of bandpass digital filters to divide the frequency spectrum of the sampled signal into  $M$  subbands. The output of each of these filters is then subsampled by a factor of  $M$  so that the number of subband samples produced is the same as the original number of time-domain samples. From the sampling theorem, because the output of each bandpass filter has a bandwidth of  $B/M$ , the sampling rate can also be reduced to  $2B/M$  while still guaranteeing perfect reconstruction.

##### 8.3.1.1. The Effect of Subsampling

Subsampling by a factor of  $M$  is effectively dividing the sampling frequency by  $M$ . This means that the sampling frequency is now not high enough to eliminate aliasing errors and the spectrum becomes distorted. This distorted spectrum is a combination of the  $M$  repetitions of the original spectrum and can be represented by the equation

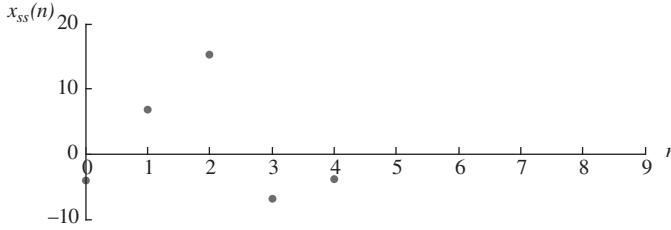
$$X_{ss}(\omega) = \frac{1}{M} \sum_{j=0}^{M-1} X_s\left(\frac{\omega + j\pi}{M}\right) \quad (8.5)$$

Equation (8.5) shows that, after filtering and subsampling, the spectrum of the outputs of the analysis filter bank are given by the equation

$$S_i(\omega) = \frac{1}{M} \sum_{j=0}^{M-1} X_s\left(\frac{\omega + j\pi}{M}\right) H_i\left(\frac{\omega + j\pi}{M}\right) \quad \text{for } i = 0, 1, 2, \dots, M-1 \quad (8.6)$$

### EXAMPLE 8.1

Consider the sampled signal  $x(n)$  with the magnitude spectrum  $|X_s(\omega)|$  as shown in Figure 8.2. Now if this signal is subsampled by a factor of 2, the result is the signal  $x_{ss}(n)$  as shown in Figure 8.4.

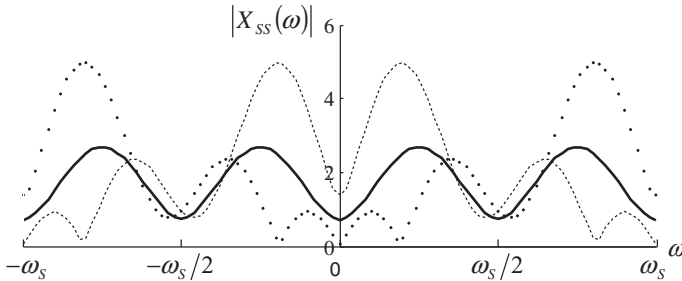


**Figure 8.4** The subsampled signal  $x_{ss}(n)$ .

However, because the sampling frequency has been halved, the aliased spectrum is given by the following equation.

$$X_{ss}(\omega) = \frac{1}{2} \left[ X_s\left(\frac{\omega}{2}\right) + X_s\left(\frac{\omega + \pi}{2}\right) \right] \quad (8.7)$$

The magnitude spectrum of this subsampled signal is shown as the full line in Figure 8.5; the dashed line is  $|X(\omega/2)|$  and the dotted line is  $|X((\omega + \pi)/2)|$ . Remember that the magnitude spectra of the two terms in Equation (8.7) cannot simply be added together. Figure 8.5 shows the magnitude spectrum of the subsampled signal after the two terms have been added as complex signals with both magnitude and phase information.



**Figure 8.5** The magnitude spectrum of the subsampled signal  $|X_{ss}(\omega)|$  (full line) with the magnitude spectrum of  $|X_s(\omega/2)|$  (dashed line) and the magnitude spectrum of  $|X_s((\omega + \pi)/2)|$  (dotted line). ■

### 8.3.2. The Synthesis Filter Bank

The synthesis stage also uses a set of bandpass filters to reconstruct the samples of the time-domain signal. The input to each of these synthesis filters is upsampled by  $M$  so that the output of each filter has the same number of samples as the original time-domain signal. This upsampling process is implemented by inserting  $M - 1$  zero-valued samples between each sample of the subsampled signal. The output samples from each synthesis filter are then added together to form the reconstructed signal.

#### 8.3.2.1. The Effect of Upsampling

Upsampling by a factor of  $M$  is effectively dividing the frequency of each component by  $M$ . The spectrum of the upsampled signal is therefore given by the following equation:

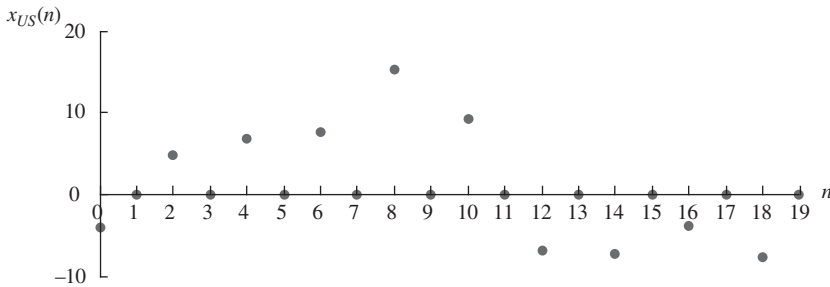
$$X_{US}(\omega) = X_S(M\omega) \quad (8.8)$$

Equation (8.8) shows that, after upsampling and filtering, the spectrum of the outputs of the synthesis filter bank in Figure 8.3 is given by the equation

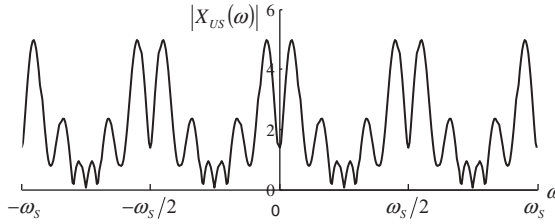
$$R_i(\omega) = \frac{1}{M} G_i(\omega) \sum_{j=0}^{M-1} X_S(\omega + j\pi) H_i(\omega + j\pi) \quad \text{for } i = 0, 1, 2, \dots, M-1 \quad (8.9)$$

#### EXAMPLE 8.2

Consider the sampled signal  $x(n)$  with the magnitude spectrum  $|X_S(\omega)|$  as shown in Figure 8.2. If this signal is upsampled by a factor of 2, the result is the signal  $x_{US}(n)$  as shown in Figure 8.6, and the magnitude spectrum of this upsampled signal is shown in Figure 8.7.



**Figure 8.6** The upsampled signal  $x_{US}(n)$ .



**Figure 8.7** The magnitude spectrum of the upsampled signal  $|X_{US}(\omega)|$ . ■

### 8.3.3. Filters for Perfect Reconstruction

The reconstructed signal from the synthesis filterbank in Figure 8.3 is found by adding the outputs of the synthesis filters and is given by

$$X'_S(\omega) = \frac{1}{M} \sum_{i=0}^{M-1} G_i(\omega) \sum_{j=0}^{M-1} X_S(\omega + j\pi) H_i(\omega + j\pi) \quad (8.10)$$

and rearranging this equation to separate the terms where  $j = 0$  gives the following equation:

$$X'_S(\omega) = \frac{1}{M} X_S(\omega) \sum_{i=0}^{M-1} G_i(\omega) H_i(\omega) + \frac{1}{M} \sum_{j=1}^{M-1} X_S(\omega + j\pi) \sum_{i=0}^{M-1} G_i(\omega) H_i(\omega + j\pi) \quad (8.11)$$

Equation (8.11) has one term that contains the original sampled signal  $X_S(\omega)$ , and the remaining terms contain frequency-shifted versions of the original signal  $X_S(\omega + j\pi)$ . To perfectly reconstruct the original signal, a set of filters that satisfy the following equations must be found:

$$\left| \sum_{i=0}^{M-1} G_i(\omega) H_i(\omega) \right| = M \quad (8.12)$$

$$\sum_{j=1}^{M-1} \sum_{i=0}^{M-1} G_i(\omega) H_i(\omega + j\pi) = 0 \quad (8.13)$$

Equation (8.12) tells us that if the frequency response for the analysis and synthesis filters for each subband are multiplied together and these combined frequency responses are added for all subbands, then the overall frequency response should be constant over the bandwidth of the original sampled signal. Equation (8.13) tells us that if the analysis filter for one subband is multiplied with the synthesis filters for all other subbands and these combined frequency responses are added for all subbands, then this overall frequency response should be zero. For a set of filters that satisfy these equations, the aliasing terms from Equation (8.11) are eliminated, and the overall frequency response of the filterbank is constant over the bandwidth of the signal.

## 8.4. COSINE-MODULATED FILTERS

One technique that can be used to determine a set of filters that satisfy the perfect reconstruction equations is to use a set of *cosine-modulated filters*. In this technique a prototype filter is determined, and each of the analysis and synthesis filters is derived from this prototype filter using cosine modulation.

For an M-channel subband filter bank, the impulse responses for the analysis and synthesis filters are given by the following equations:

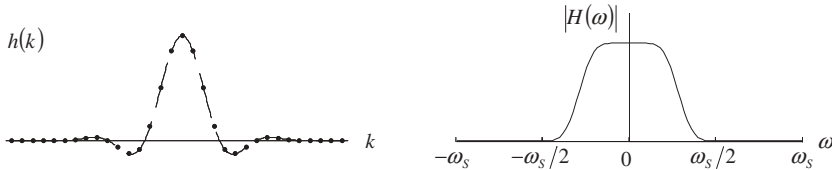
$$h_i(k) = h(k) \cos\left(\frac{(2i+1)(k - \frac{M}{2})\pi}{2M}\right) \quad (8.14)$$

$$g_i(k) = h(k) \cos\left(\frac{(2i+1)(k + \frac{M}{2})\pi}{2M}\right) \quad (8.15)$$

where  $h(k)$  is the impulse response of the prototype filter.

### EXAMPLE 8.3

The impulse response  $h(k)$  of a prototype filter for a two-channel filterbank is shown in Figure 8.8(a), and the magnitude of the frequency response of this filter  $|H(\omega)|$  is shown in Figure 8.8(b).



**Figure 8.8** (a) The impulse response  $h(k)$  and (b) the magnitude of the frequency response  $|H(\omega)|$  of a prototype filter for a two-channel perfect reconstruction filterbank.

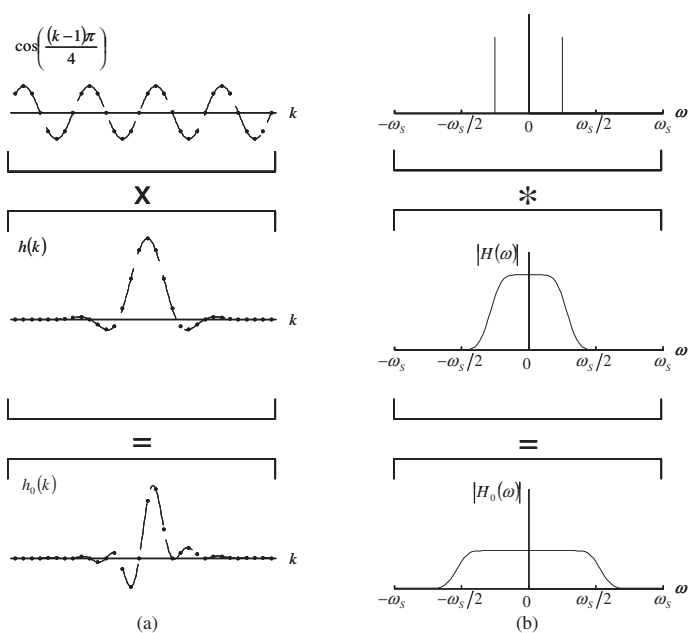
The two analysis filters are determined using the following equations:

$$h_0(k) = h(k) \cos\left(\frac{(k-1)\pi}{4}\right) \quad (8.16)$$

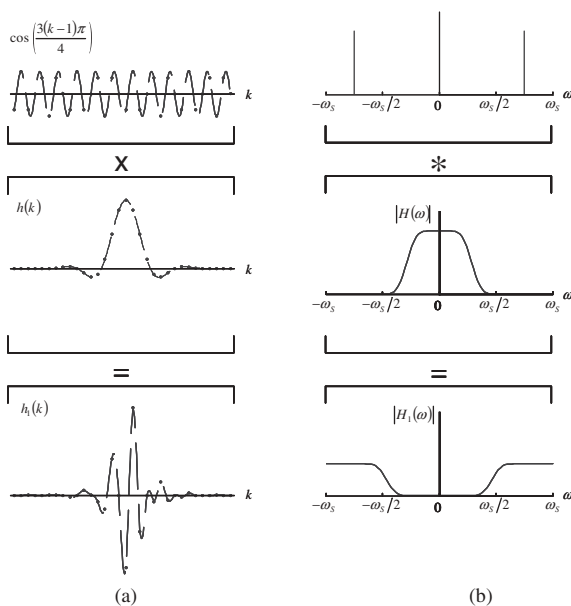
$$h_1(k) = h(k) \cos\left(\frac{3(k-1)\pi}{4}\right) \quad (8.17)$$

The filter coefficients of the analysis filters are found by multiplying the coefficients of the prototype filter by a sampled cosine wave as shown in Figure 8.9(a). In the frequency domain this is equivalent to the convolution of the spectrum of the prototype filter with the spectrum of the cosine wave as shown in Figure 8.9(b).

Figure 8.10 shows the same process for  $h_1(k)$ , and Figures 8.9 and 8.10 show that the two analysis filters divide the spectrum of the sampled input signal into two equal width subbands. ■



**Figure 8.9** (a) The coefficients of the low-pass filter are found by multiplying the prototype filter coefficients by a sampled cosine wave. (b) In the frequency domain, this is equivalent to the convolution of the prototype frequency response with an impulse function at the frequency of the cosine.



**Figure 8.10** (a) The coefficients of the high-pass filter are found by multiplying the prototype filter coefficients by a sampled cosine wave. (b) In the frequency domain this is equivalent to the convolution of the prototype frequency response with an impulse function at the frequency of the cosine.

The two synthesis filters are determined using the following equations:

$$g_0(k) = h(k) \cos\left(\frac{(k+1)\pi}{4}\right) \quad (8.18)$$

$$g_1(k) = h(k) \cos\left(\frac{3(k+1)\pi}{4}\right) \quad (8.19)$$

The sampled cosine waves and the resulting filter coefficients for the synthesis filters are shown in Figure 8.11. Note from Figure 8.11 that the synthesis filters are simply the analysis filters reversed in time.

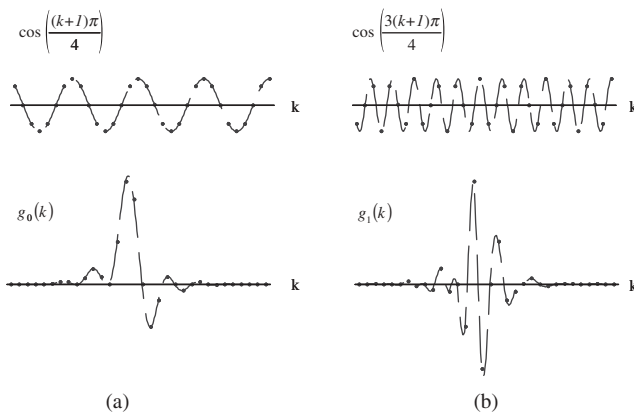
The output of each of the analysis filters is the digital convolution of the impulse response of the filters with the input signal that is given by the following equations:

$$s_0(n) = \sum_{k=0}^N h_0(k) x(n-k) \quad (8.20)$$

$$s_1(n) = \sum_{k=0}^N h_1(k) x(n-k) \quad (8.21)$$

A simple implementation of an  $M$ -channel analysis filterbank with filters of  $N$  coefficients requires  $MN$  multiplications and  $M(N-1)$  additions for each sample of the input signal.

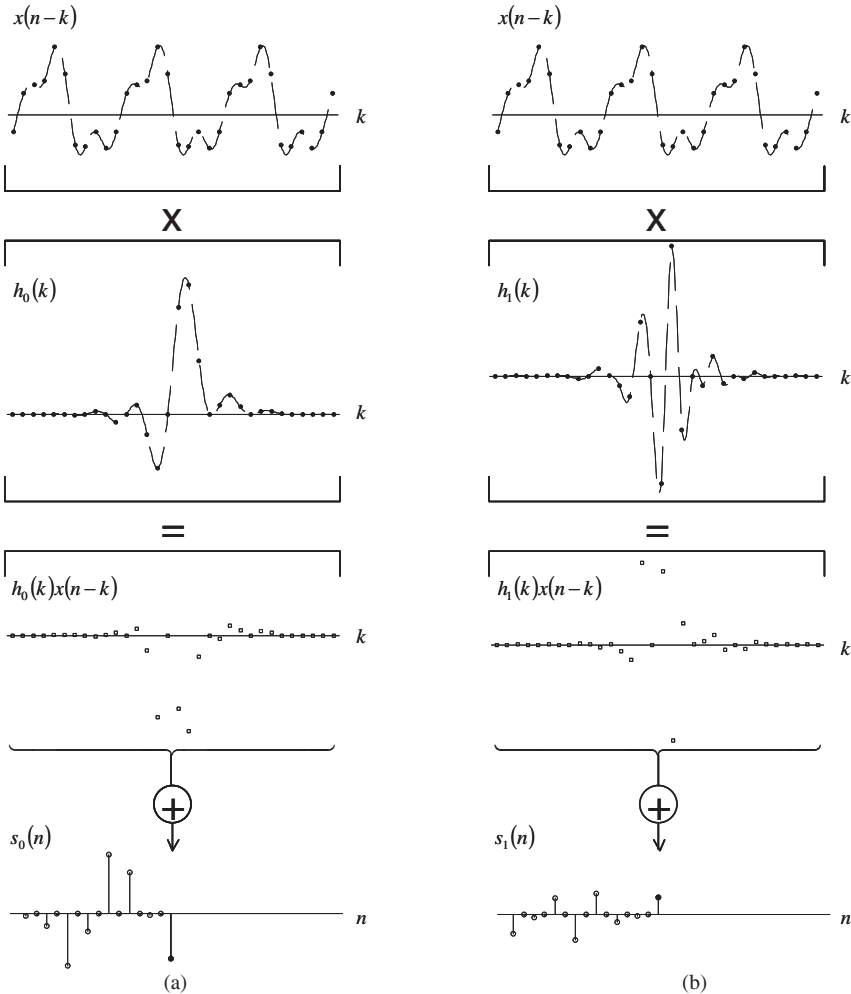
Similarly, the output of each of the synthesis filters is the digital convolution of the impulse response of the filters with the upsampled subband samples. The outputs of these filters are then added to produce the reconstructed output samples. A simple implementation of an  $M$ -channel synthesis filterbank with filters of  $N$  coefficients requires  $MN$  multiplications and  $MN-1$  additions for each sample of the reconstructed signal.



**Figure 8.11** The sampled cosine waves and the resulting filter coefficients for (a) the low-pass synthesis filter and (b) the high-pass synthesis filter.

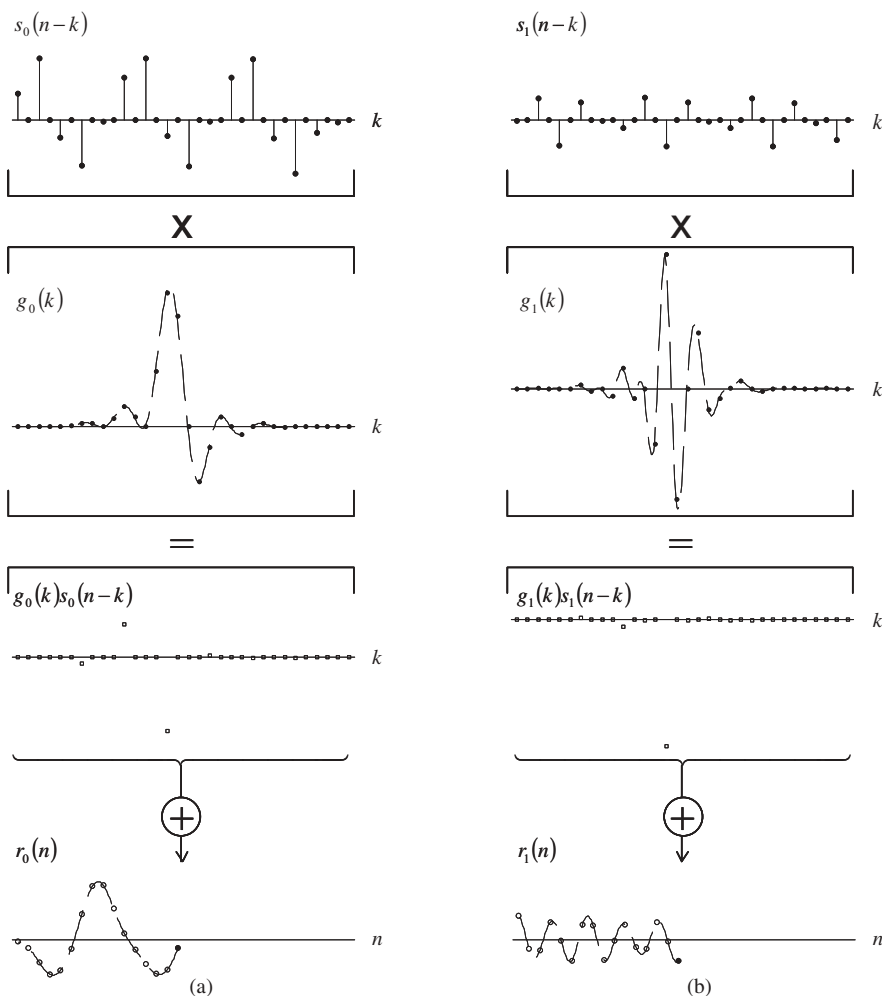
**EXAMPLE 8.4**

Figure 8.12(a) shows a graphical representation of the calculations required for an output subband sample of the low-pass filter in the analysis filter bank. The filter has a length of 32, so each set of 32 input samples is first reversed in time and then multiplied by the corresponding value of the impulse response. The sum of all these product terms is then calculated to obtain the single output subband sample  $s_0(n)$  (shown as a full circle). For a two-channel filterbank using the analysis filters from the previous example, each output subband sample requires 32 multiplications and 31 additions. These same calculations are also required for the high-pass filter. For each input sample, a total of 64 multiplications and 62 additions are required to obtain each pair of low-pass and high-pass subband samples.



**Figure 8.12** Multiplications and additions required for one output sample of the analysis filter (shown as a filled circle). (a) Low-pass filter and (b) high-pass filter (zero samples have been added to the output sample sequence to maintain the same timescale as for the input samples).

Figure 8.13(a) shows a graphical representation of the calculations required for an output sample of the low-pass filter in the synthesis filter bank (shown as a full circle). The input to the filter,  $s_0(n-k)$ , is the set of 32 time-reversed subband samples from the low-pass analysis filter that have been subsampled by a factor of 2 and then upsampled by inserting a zero-valued sample between each sample. Each set of 32 samples of  $s_0(n-k)$  is multiplied by the corresponding value of the impulse response of the low-pass synthesis filter  $g_0(k)$ . Then the sum of all these product terms is calculated to obtain the single reconstructed low-pass output sample  $r_0(n)$ . For a two-channel filter bank using the synthesis filters from the previous example, each reconstructed output sample requires 32 multiplications and 31 additions. These same calculations are also required for the high-pass filter, and a further addition is required to add the low-pass and high-pass samples together. Therefore, for each output sample, a total of 64 multiplications and 63 additions are required.



**Figure 8.13** Multiplications and additions required for one output sample of the synthesis filter (shown as a filled circle). (a) Low-pass filter and (b) high-pass filter. ■

## 8.5. EFFICIENT IMPLEMENTATION OF A COSINE-MODULATED FILTERBANK

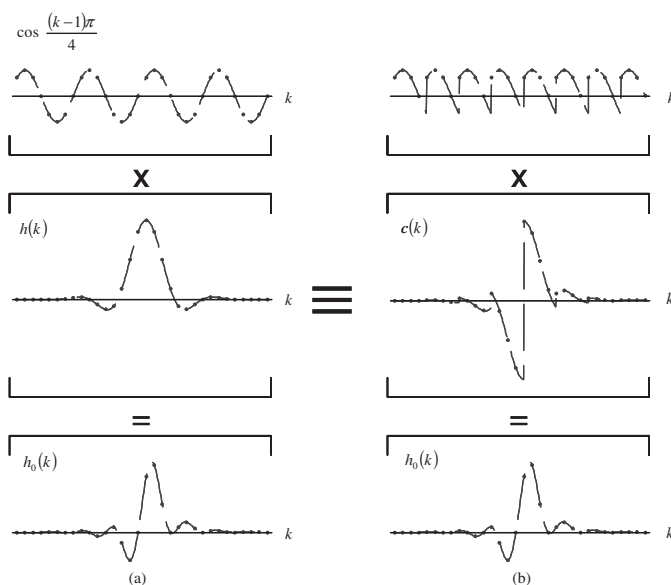
The standard implementation of a subband filterbank requires many more calculations than is necessary. A more efficient implementation of a set of cosine-modulated filters was proposed by Joseph Rothweiler in 1983 [1]. This efficient implementation takes advantage of two things: (a) the prototype filter coefficients are common to all filters with the only difference being the frequency of the sampled cosine waves and (b) the periodic nature of the sampled cosine waves.

### 8.5.1. Analysis Filter

In order to explain the basic principles behind Rothweiler's proposal, first consider the following example that shows how the two-channel analysis filterbank from Section 8.4 can be implemented more efficiently.

#### EXAMPLE 8.5

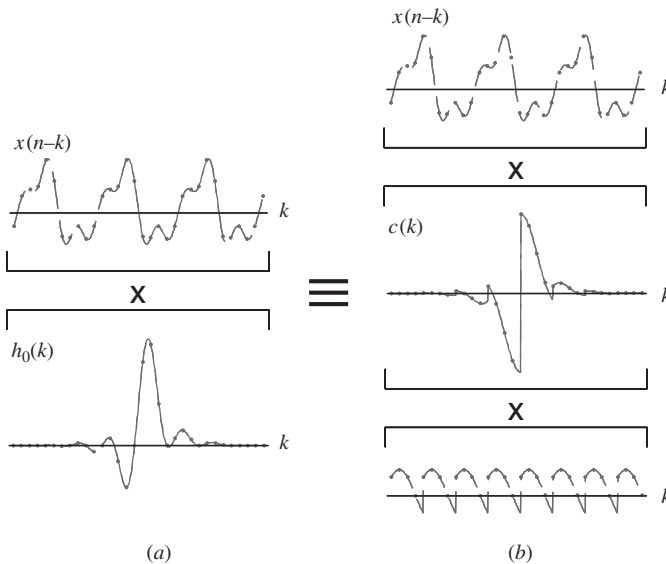
Figure 8.14(a) shows the conventional method for calculating the low-pass filter impulse response by multiplying the prototype filter impulse response with a sampled cosine wave. Figure 8.14(b) shows how the same impulse response can be calculated by inverting every second set of four samples in both the cosine wave and the prototype filter response. By inverting every second set of four samples, the sampled cosine wave becomes a repeating set



**Figure 8.14** (a) The conventional approach to implementing the analysis filter. (b) In Rothweiler's approach the cosine-modulated filters can be constructed by multiplying  $c(k)$  by a repeating set of samples from the original sampled cosine wave.

of the same four samples of the original wave and the prototype filter impulse response,  $h(k)$ , becomes the *modified impulse response*,  $c(k)$ .

In the discrete convolution process that would take place in a conventional implementation of the analysis filterbank, 32 time-reversed samples of the audio input are multiplied with the 32 samples of the low-pass filter impulse response as shown in Figure 8.15(a). This is equivalent to multiplying by the modified impulse response,  $c(k)$ , and then by the repeating set of the same four samples of the original cosine wave as shown in Figure 8.15(b).

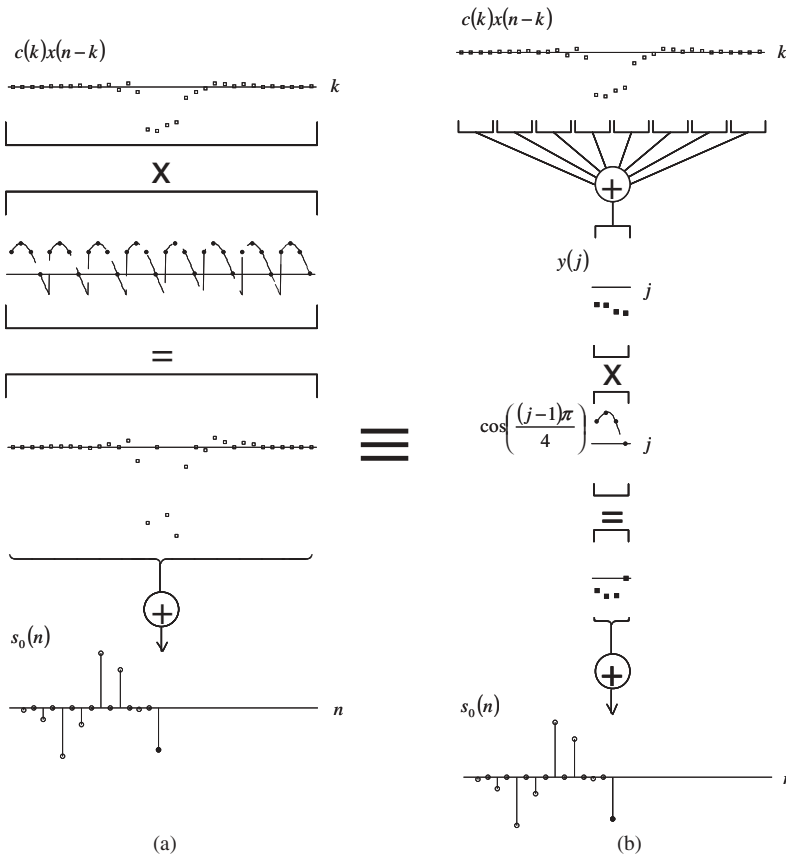


**Figure 8.15** (a) The conventional approach to filtering the input signal involves multiplying by the low-pass filter impulse response. (b) In Rothweiler's approach, multiplying by the low-pass filter impulse response is equivalent to multiplying by  $c(k)$  and then by a repeating set of samples from the original sampled cosine wave.

If  $c(k)$  is substituted for  $h(k)$ , the next step in the convolution process is simplified. The corresponding samples of  $c(k)x(n-k)$  are simply multiplied by the repeating set of four cosine samples, and then the sum of these products is calculated to determine the current output sample value as shown in Figure 8.16(a). However, in the calculations performed in Figure 8.16(a), all samples of  $c(k)x(n-k)$  that are four samples apart are multiplied by the same cosine value. This set of four cosine samples can be factored out as shown in Figure 8.16(b). To achieve this factorization, the 32 samples of  $c(k)x(n-k)$  are divided into eight groups of four samples, and the corresponding samples in each of these groups are added to determine the intermediate sum,  $y(j)$ . Each sample of this intermediate sum is then multiplied by its corresponding cosine value, and the sum of these products is calculated to determine the current output sample value.

In this approach, instead of performing 32 multiplications and 31 additions, 32 multiplications and 28 additions are performed to calculate the intermediate sum, and then further four multiplications and three additions are performed to calculate the output value.

For the low-pass filter on its own, this approach does not produce any significant saving in calculations, but there are more savings to be made for the entire analysis filterbank. Because the only difference between the values of the low-pass and high-pass impulse response is the

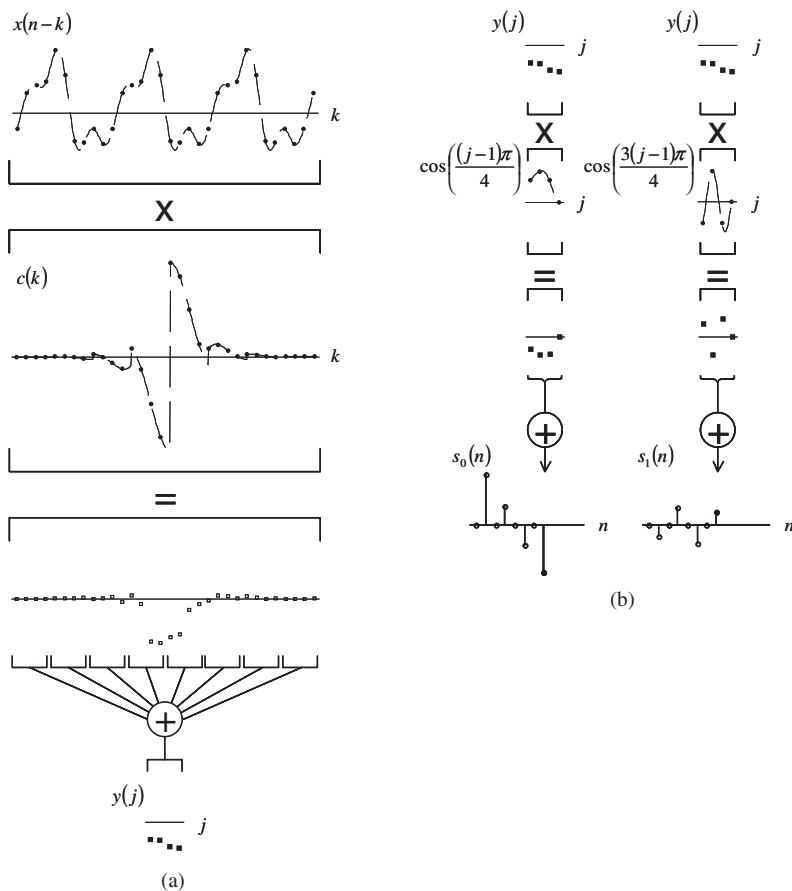


**Figure 8.16** (a) The corresponding samples of  $c(k)x(n-k)$  are simply multiplied by the repeating set of four cosine samples, and then the sum of these products is calculated to determine the current output sample value (shown as a full circle). (b) The process shown in (a) is simplified by dividing the 32 samples of  $c(k)x(n-k)$  into eight groups of four samples, and the corresponding samples in each of these groups are added to determine the intermediate sum,  $y(j)$ . Each sample of this intermediate sum is then multiplied by its corresponding cosine value, and the sum of these products is calculated to determine the current output sample value (shown as a full circle).

frequency of the sampled cosine wave, the intermediate sum is reused in the calculations for the output sample of the high-pass filter. The four samples of the low-pass cosine wave in Figure 8.16(b) are replaced with the first four samples of the high-pass cosine wave to determine the current output sample for the high-pass filter.

Figure 8.17 shows the efficient implementation of the two-channel analysis filter bank from the previous examples using Rothweiler's method. Figure 8.17(a) shows the calculation of the intermediate sum, and Figure 8.17(b) shows how this intermediate sum is multiplied by four samples from both the low-pass and high-pass cosine waves to obtain the output samples of the low-pass and high-pass filters, respectively.

Instead of the 64 multiplications and 62 additions required in a conventional implementation of this filterbank, 32 multiplications and 28 additions are required for the intermediate sum, and then four multiplications and three additions are required for each of the low-pass



**Figure 8.17** (a) In Rothweiler's efficient implementation of a two-channel cosine-modulated analysis filterbank, the product of the time-reversed input samples,  $x(n-k)$ , and the modified impulse response,  $c(k)$ , is divided into eight groups of four samples, and the samples in each of these groups are added to determine the intermediate sum,  $y(j)$ . (b) Each sample of this intermediate sum is then multiplied by its corresponding high-pass and low-pass cosine values, and the sum of these products is calculated to determine the current high-pass and low-pass output sample values (shown as a full circle).

and high-pass outputs giving a total of 40 multiplications and 34 additions. It is clear from this example that Rothweiler's approach becomes much more efficient than a conventional implementation as the number of bandpass filters in the filterbank increases. ■

For the general case of an  $M$ -channel filterbank using a prototype filter with length  $N$ , the following MATLAB code defines the operations performed in the analysis filter bank. The array  $x$  acts as a FIFO buffer to hold the audio samples in time-reversed order; the modified impulse response is contained in the array  $c$ , and the set of  $M$  PCM samples to be converted to subband samples is contained in the

array `next_pcm_input`. The array `subband_samples` contains the set of  $M$  subband samples corresponding to the current set of  $M$  PCM samples.

```
% shift in the next M audio samples in time-reversed order
for k = N:-1:M+1
    x(k) = x(k-M);
end
for k = M:-1:1
    x(k) = next_pcm_input(M-k+1);
end

% multiply the audio samples by the modified impulse response c(k)
for k = 1:N
    z(k) = c(k)*x(k);
end

% calculate the intermediate sum y(j)
for j = 1:2*M
    y(j) = 0;
    for q = 0:N/(2*M)-1;
        y(j) = y(j)+z(j+2*M*q);
    end
end

% multiply the intermediate sum by each sampled cosine wave
% to produce the next M subband samples
for i = 1:M;
    subband_samples(i) = 0;
    for j = 1:2*M
        basis(j,i) = cos(((2*i-1)*(j-(M/2)-1)*pi)/(2*M));
        subband_samples(i) = subband_samples(i)+y(j)*basis(j,i);
    end
end
```

This implementation requires  $N$  multiplications and  $N - 2M$  additions to calculate  $y$  and then  $2M^2$  multiplications and  $2M^2 - M$  additions to calculate the output samples of the  $M$  analysis filters.

In the analysis filterbank used in the MPEG audio coding algorithms, the number of bandpass filters in the filterbank is 32, and the prototype impulse response has a length of 512, that is,  $M = 32$  and  $N = 512$ . A conventional implementation would therefore require  $512 \times 32 = 16,384$  multiplications and  $511 \times 32 = 16,352$  additions for each set of 32 output samples. An implementation using Rothweiler's method requires 512 multiplications and 448 additions to calculate the intermediate sum and then 64 multiplications and 63 additions for each of the 32 output samples, giving a total of 2560 multiplications and 2464 additions.

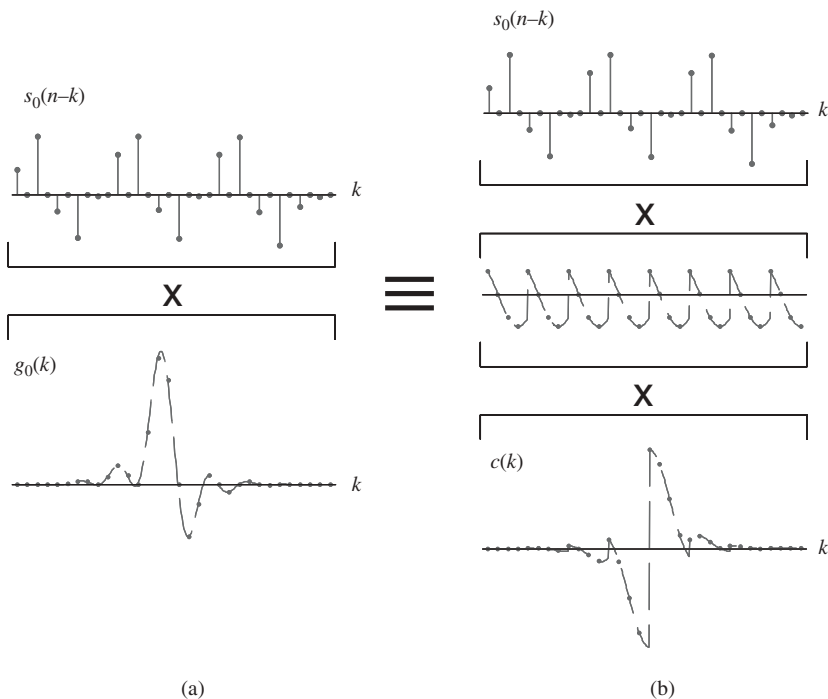
In addition, the equation for calculating the  $M$  subband samples is effectively describing a modified discrete cosine transform (DCT) with  $2M$  inputs and  $M$  outputs, so a further reduction in computational complexity is possible by using a fast DCT implementation of this equation.

### 8.5.2. Synthesis Filter

As for the analysis filter, to explain Rothweiler's proposal it is best to first consider the following example that shows how the two-channel synthesis filterbank from Section 8.4 is implemented more efficiently.

#### EXAMPLE 8.6

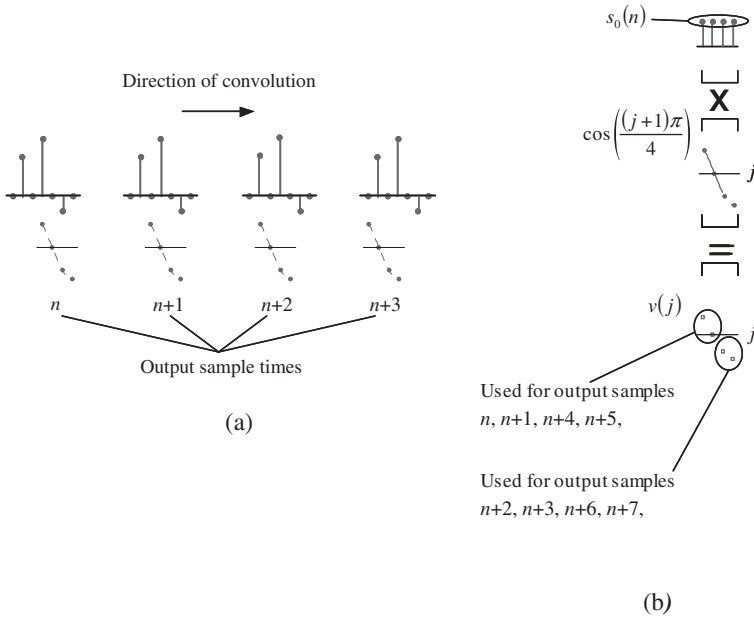
In the discrete convolution process that takes place in a conventional implementation of the synthesis filterbank, 32 time-reversed subband samples are multiplied with the 32 samples of the low-pass filter impulse response as shown in Figure 8.18(a). This is equivalent to multiplying by a repeating set of the same four samples of the original cosine wave and then by the modified impulse response,  $c(k)$ , as shown in Figure 8.18(b).



**Figure 8.18** (a) The conventional approach to reconstructing the time-domain signal involves multiplying by the low-pass synthesis filter impulse response. (b) In Rothweiler's approach, multiplying by the low-pass filter impulse response is equivalent to multiplying by a repeating set of samples from the original sampled cosine wave and then by  $c(k)$ .

During the course of the process shown in Figure 8.18(b), each subband sample is only multiplied by four different cosine sample values. To explain this further, consider the diagram in Figure 8.19(a) that shows the relative position of the subband samples and the first four samples of the cosine wave for four consecutive output sample times. This diagram

shows that, because every second subband sample is zero, the same set of input subband samples is used in the convolution process for the output samples at times  $n$  and  $n + 1$ . The only difference between times  $n$  and  $n + 1$  is the relative position of the subband samples and the cosine samples. Then for output samples at times  $n + 2$  and  $n + 3$ , the next subband sample is included in the convolution process, and this new set of subband samples is used for both output sample times. Hence, during the entire convolution process, one individual subband sample is multiplied by the first two cosine samples for times  $n$  and  $n + 1$ , the last two cosine samples for times  $n + 2$  and  $n + 3$ , the first two cosine samples for times  $n + 4$  and  $n + 5$ , the last two cosine samples for times  $n + 6$  and  $n + 7$ , and so on.

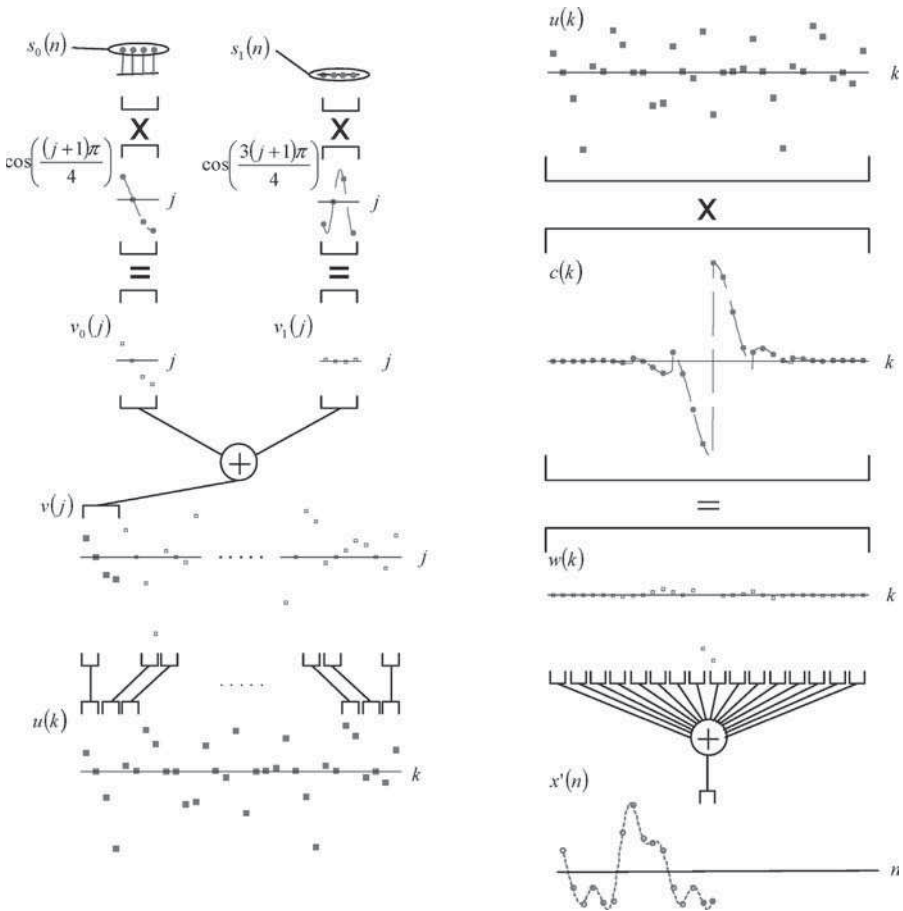


**Figure 8.19** (a) One individual subband sample during the convolution process is multiplied by the first two repeating cosine samples for times  $n$  and  $n + 1$  and the last two cosine sample values for times  $n + 2$  and  $n + 3$ . (b) The product of one individual subband sample with each of the four cosine sample values is denoted by  $v(j)$ . The first two values of  $v(j)$  are included in the convolution process for output samples at times  $n, n + 1, n + 4, n + 5, \dots$ , and the last two values are included in the convolution process for output samples at times  $n + 2, n + 3, n + 6, n + 7, \dots$

Now consider the product of one individual subband sample with each of the four cosine sample values,  $v(j)$ , as shown in Figure 8.19(b). The first two values of  $v(j)$  are included in the convolution process for output samples at times  $n$  and  $n + 1$ , and the last two values are included in the convolution process for output samples at times  $n + 2$  and  $n + 3$ . Then for output samples at times  $n + 4$  and  $n + 5$ , the first two values of  $v(j)$  are reused, and for output samples at times  $n + 6$  and  $n + 7$ , the last two values of  $v(j)$  are reused, and so on.

For the case of the two-channel synthesis filter from the previous example, the number of calculations required is reduced by first calculating  $v_0(j)$  using the first four low-pass cosine samples and  $v_1(j)$  using the first four high-pass cosine samples as shown in Figure 8.20(a). The corresponding values of  $v_0(j)$  and  $v_1(j)$  are then added to produce four intermediate values,  $v(j)$ . For each two input subband samples, four new  $v(j)$  values are produced. The

current set of output samples only requires the first two values, but the last two  $v(j)$  values are needed for the next two output samples. To save recalculating values, a FIFO buffer is constructed, and each time four new values for  $v(j)$  are calculated, the current contents of the buffer is shifted by four positions and the current set of  $v(j)$  values is placed at the start of the buffer. Note that there are 64 values stored in the FIFO buffer for any one output sample time, so the complete set of values in the buffer is not shown in Figure 8.20(a).



**Figure 8.20** (a) In Rothweiler's efficient implementation of a two-channel cosine-modulated synthesis filterbank,  $v_0(j)$  is calculated using the first four low-pass cosine samples and  $v_1(j)$  using the first four high-pass cosine samples. The corresponding values of  $v_0(j)$  and  $v_1(j)$  are then added to produce four intermediate values,  $v(j)$ . A FIFO buffer is constructed and, each time four new values for  $v(j)$  are calculated, the current contents of the buffer is shifted by four positions, and the current set of  $v(j)$  values is placed at the start of the buffer. The set of values  $u(k)$  is constructed by taking the first two values from the current set of four  $v(j)$  values, the second two samples from the previous set of four  $v(j)$  values, and so on. (b) The values of  $u(k)$  are multiplied by the corresponding samples of the modified prototype filter  $c(k)$  to give  $w(k)$ . The even values in  $w(k)$  are added together to produce the output sample at time  $n$ , and the odd values are added together to produce the output sample at time  $n + 1$  (shown as full circles).

The set of values  $u(k)$  is then constructed as shown in Figure 8.20(a) by taking the first two values from the current set of four  $v(j)$  values, the second two samples from the previous set of four  $v(j)$  values, and so on.

The values of  $u(k)$  are then multiplied by the corresponding samples of the modified prototype filter  $c(k)$  to give  $w(k)$  as shown in Figure 8.20(b). The even values in  $w(k)$  are then added together to produce the output sample at time  $n$ , and the odd values are added together to produce the output sample at time  $n + 1$ .

Instead of the 64 multiplications and 63 additions required in a conventional implementation of this filterbank, four multiplications and three additions are required for each of the low-pass and high-pass outputs, four additions are required for the intermediate values of  $v(j)$ , and 32 multiplications and 30 additions are required for the two output sample values giving a total of 40 multiplications and 40 additions. Again, it is clear from this example that Rothweiler's approach becomes much more efficient than a conventional implementation as the number of bandpass filters in the synthesis filterbank increases. ■

For the more general case of an  $M$ -channel filterbank using a prototype filter with length  $N$ , the following MATLAB code defines the operations performed in the synthesis filter bank. The array  $v$  acts as a FIFO buffer to hold intermediate values  $v(j)$ ; the modified impulse response is contained in the array  $c$ , and the set of  $M$  subband samples to be converted to PCM samples is contained in the array `subband_samples`. The array `next_pcm_output` contains the set of  $M$  output PCM samples corresponding to the current set of  $M$  subband samples.

```
% shift the values in the FIFO buffer v(j) by 2M elements
for j = 2*N:-1:2*M+1
    v(j) = v(j-2*M);
end

% calculate the next 2M values and place them at the start of the FIFO buffer v(j)
for j = 1:2*M
    v(j) = 0;
    for i = 1:M
        basis(i,j) = cos(((2*i-1)*(j+M/2-1)*pi)/(2*M));
        v(j) = v(j) + subband_samples(i)*basis(i,j);
    end
end

% calculate the intermediate sum u(k)
for q = 0:N/(2*M)-1
    for i = 1:M
        u(2*M*q+i) = v(4*M*q+i);
        u(2*M*q+M+i) = v(4*M*q+3*M+i);
    end
end

% multiply the intermediate sum by the modified impulse response c(k)
for k = 1:N
    w(k) = u(k)*c(k);
end

% calculate the next M output PCM samples
```

```

for n = 1:M
    next_pcm_output(n) = 0;
    for p = 0:N/M-1
        next_pcm_output(n) = next_pcm_output(n)+w (n+M*p)*M;
    end
end
end

```

It is important to note that for each  $M$  output samples of the synthesis filterbank, it is only necessary to calculate the first  $2M$  values of  $v(j)$  because the remaining values have already been calculated for previous output samples of the filterbank.

This implementation requires  $2M^2$  multiplications and  $2M^2 + M$  additions to calculate the first  $2M$  samples of  $v$  and then  $N$  multiplications and  $N - M$  additions to calculate the  $M$  samples of the reconstructed signal.

In the synthesis filterbank used in the MPEG audio coding algorithms, the number of bandpass filters in the filterbank is 32, and the prototype impulse response has a length of 512, that is,  $M = 32$  and  $N = 512$ . Therefore, a conventional implementation would require  $512 \times 32 = 16,384$  multiplications and  $512 \times 32 - 1 = 16,383$  additions for each set of 32 output samples. But an implementation using Rothweiler's method requires 2048 multiplications and 2080 additions to calculate the first 64 samples of  $v(j)$  and then 512 multiplications and 480 additions to calculate the 32 samples of the reconstructed signal, giving a total of 2560 multiplications and 2560 additions.

In addition, the equation for calculating each  $2M$  values of  $v(j)$  is effectively describing a modified inverse DCT with  $M$  inputs and  $2M$  outputs, so a further reduction in computational complexity is possible by using a fast IDCT implementation of this equation.

## 8.6. TIME-DOMAIN ALIASING CANCELLATION

An alternative method to conducting subband filtering with aliasing cancellation in the frequency domain is to construct a set of filters that overlap in both time and frequency and provide aliasing cancellation in the time domain. This system was first proposed by John Princen and Alan Bradley in 1986 [2] and is commonly referred to as time-domain aliasing cancellation (TDAC).

This system is effectively a transform-based approach to conversion between the time and frequency domains. However, with TDAC transforms a critically sampled system is designed using overlapping transform windows in the time domain. In all filterbank designs there is some trade-off between window size in the time domain and resolution in the frequency domain. The main disadvantage of orthonormal transform-based techniques is that the number of samples in the time-domain corresponds directly to the number of frequency-domain coefficients. This means that relatively small time-domain windows are used, and the resulting coefficients represent strongly overlapping subbands in the frequency domain. One solution to this

problem is to use a large strongly overlapping window in the time domain such as that used in the cosine-modulated filter bank of the previous section. The outputs of this filterbank are then subsampled, and aliasing is introduced in the frequency domain. If the frequency responses of these filters are designed correctly, a perfect reconstruction filter bank is produced and the frequency-domain aliasing can be removed.

Time-domain aliasing cancellation takes a different approach. In this system an overlapped window in the time domain is used, but this window typically only overlaps by half the size of the window. By extending the size of the window, the amount of overlap in the corresponding frequency-domain subbands is reduced compared to a critically sampled transform. However, there are now more frequency-domain coefficients than time-domain samples in the analysis stage. To overcome this problem, in a TDAC system, only half the frequency-domain coefficients are transmitted, and distortion is introduced into the reconstructed time-domain samples. If the transform is designed correctly, the distortion introduced by discarding coefficients is directly canceled out by overlapping and adding successive windows in the time domain.

Such a TDAC system uses certain properties of the DCT to guarantee perfect reconstruction. The general equation for an  $N$ -point DCT of the sequence  $x(n)$  is given by

$$X(k) = \sum_{n=0}^{N-1} x(n) \cos\left(\frac{2\pi k}{N}(n + n_0)\right), \quad \text{for } k = 0 \dots N-1 \quad (8.22)$$

and the inverse of this transform is given by

$$x'(r) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) \cos\left(\frac{2\pi k}{N}(r + n_0)\right), \quad \text{for } r = 0 \dots N-1 \quad (8.23)$$

This set of equations produces a distorted version of the original sequence. Note that the variable  $r$  has been substituted for the variable  $n$  to indicate the sample time of the outputs of the inverse transform. This substitution is necessary for the remainder of the derivation as the equations for the forward and inverse transforms are combined.

An example of the type of distortion produced is shown by substituting Equation (8.22) into Equation (8.23) to give

$$x'(r) = \frac{1}{N} \sum_{k=0}^{N-1} \sum_{n=0}^{N-1} x(n) \cos\left(\frac{2\pi k}{N}(n + n_0)\right) \cos\left(\frac{2\pi k}{N}(r + n_0)\right) \quad (8.24)$$

This equation can be rearranged using the simple trigonometric identity,  $\cos(A)\cos(B) = 1/2 [\cos(A - B) + \cos(A + B)]$ , to give

$$x'(r) = \frac{1}{2N} \sum_{n=0}^{N-1} x(n) \left[ \sum_{k=0}^{N-1} \cos\left(\frac{2\pi k}{N}(n - r)\right) + \sum_{k=0}^{N-1} \cos\left(\frac{2\pi k}{N}(n + r + 2n_0)\right) \right] \quad (8.25)$$

The following equation holds for the general form of the terms in the square brackets

$$\sum_{k=0}^{N-1} \cos\left(\frac{2\pi k}{N} n\right) = \begin{cases} N, & \text{if } n = 0, N, 2N, \dots \\ 0, & \text{otherwise} \end{cases} \quad (8.26)$$

Because the values of  $n$  in Equation (8.25) are restricted to the range  $0 \dots N-1$ , the first term in the square brackets in Equation (8.25) reduces to a value of  $N$  when  $n = r$  (i.e., when  $n - r = 0$ ), and the second term reduces to a value of  $N$  when  $n = N - r - 2n_0$  (i.e., when  $n + r + 2n_0 = N$ ). Hence Equation (8.25) can be rewritten as

$$x'(r) = \frac{1}{2} [x(r) + x(N - r - 2n_0)] \quad (8.27)$$

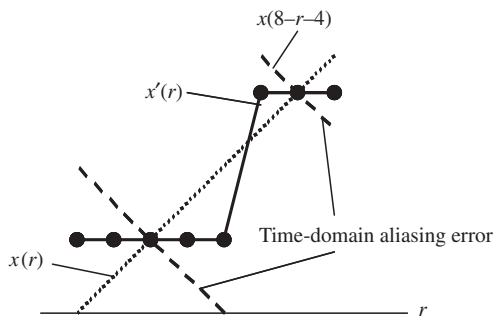
From Equation (8.27), it is evident that the distorted output signal is the sum of the original sequence and a time-reversed and shifted replica of the original sequence. This time-reversed replica of the original sequence is effectively a time-domain aliased version of the original signal.

### EXAMPLE 8.7

Determine the output signal obtained from the general DCT described in Equations (8.22) and (8.23) when the input signal is  $x(n) = [0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7]$  and the phase term is  $n_0 = 2$ . Verify that this output signal is equal to  $x'(r) = 1/2 [x(r) + x(N - r - 2n_0)]$ .

**SOLUTION** If Equations (8.22) and (8.23) are implemented directly, the output signal is given by  $x'(r) = [2 \ 2 \ 2 \ 2 \ 2 \ 6 \ 6 \ 6]$ . Then if  $N = 8$ , the second term of Equation (8.27) is given by  $x(8 - r - 4) = [4 \ 3 \ 2 \ 1 \ 0 \ 7 \ 6 \ 5]$ , and substituting these values into Equation (8.27) gives the desired result.

Figure 8.21 shows a graphical representation of the transform process with the original signal shown as the dotted line, the time-domain aliasing error as the dashed line, and the output signal of the transform process as the full line with dots showing the actual sample values.



**Figure 8.21** The output signal of the transform process is the sum of the original signal and the time-domain aliasing error. ■

Now, using this same basic idea, the general equation of the DCT is modified to give the following transform equations:

$$X(k) = \frac{2}{N} \sum_{n=0}^{N-1} x(n) \cos\left(\frac{2\pi}{4N}(2k+1)\left(2n + \frac{N}{2} + 1\right)\right), \quad \text{for } k = 0 \cdots N/2 - 1 \quad (8.28)$$

and

$$x'(r) = \sum_{k=0}^{N/2-1} X(k) \cos\left(\frac{2\pi}{4N}(2k+1)\left(2r + \frac{N}{2} + 1\right)\right), \quad \text{for } r = 0 \cdots N - 1 \quad (8.29)$$

Note that this modified transform only requires half as many transform coefficients as there are samples of the input signal. In this way, the transform is implemented on an extended set of input samples, but a critically sampled system is still maintained. Then substitute Equation (8.28) into Equation (8.29) and rearrange using the same simple trigonometric identity to give the following equation:

$$\begin{aligned} x'(r) = \frac{2}{N} \sum_{n=0}^{N-1} x(n) & \left[ \sum_{k=0}^{N/2-1} \cos\left(\frac{2\pi}{4N}(2k+1)(2n-2r)\right) \right. \\ & \left. + \sum_{k=0}^{N/2-1} \cos\left(\frac{2\pi}{4N}(2k+1)(2n+2r+N+2)\right) \right] \end{aligned} \quad (8.30)$$

The following equation holds for the general form of the terms in the square brackets:

$$\sum_{k=0}^{N/2-1} \cos\left(\frac{2\pi}{4N}(2k+1)2n\right) = \begin{cases} N/2, & \text{if } n = 0, 2N, 4N, \dots \\ -N/2, & \text{if } n = N, 3N, 5N, \dots \\ 0, & \text{otherwise} \end{cases} \quad (8.31)$$

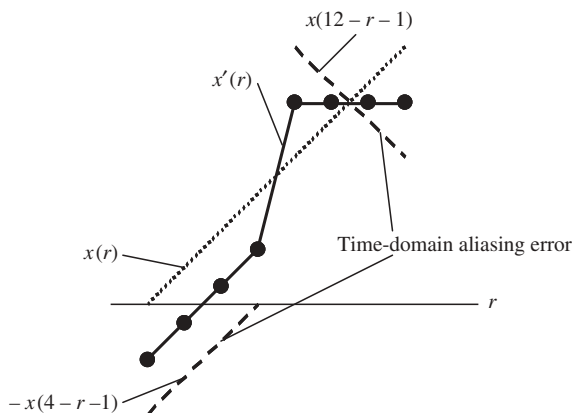
Because the values of  $n$  in Equation (8.30) are restricted to the range  $0 \cdots N - 1$ , the first term in the square brackets in Equation (8.30) reduces to a value of  $N/2$  when  $n = r$ . However, the second term in the square brackets has two different values depending on the value of  $r$ . When  $r = 0 \cdots N/2 - 1$ , the second term has the value  $-N/2$  because this range of values of  $r$  corresponds to  $n = N/2 - 1 \cdots 0$  when the equation  $n + r + N/2 + 1 = N$  is satisfied. Similarly, when  $r = N/2 \cdots N - 1$ , the second term has the value  $N/2$  because this range of values of  $r$  corresponds to  $n = N - 1 \cdots N/2$  when the equation  $n + r + N/2 + 1 = 2N$  is satisfied. Hence Equation (8.30) can be rewritten as

$$x'(r) = \begin{cases} \frac{1}{2}[x(r) - x(N/2 - r - 1)], & \text{for } r = 0 \cdots N/2 - 1 \\ \frac{1}{2}[x(r) + x(3N/2 - r - 1)], & \text{for } r = N/2 \cdots N - 1 \end{cases} \quad (8.32)$$

**EXAMPLE 8.8**

Determine the output signal obtained from the DCT described in Equations (8.28) and (8.29) when the input signal is  $x(n) = [0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7]$ . Verify that this output signal is equal to the signal obtained using Equation (8.32).

**SOLUTION** If Equations (8.28) and (8.29) are implemented directly, the output signal is given by  $x'(r) = [-1.5 \ -0.5 \ 0.5 \ 1.5 \ 5.5 \ 5.5 \ 5.5 \ 5.5]$ . Then if  $N = 8$  the second term of Equation (8.32) is given by  $-x(4 - r - 1) = [-3 \ -2 \ -1 \ 0]$  for  $r = 0 \cdots 3$  and  $x(12 - r - 1) = [7 \ 6 \ 5 \ 4]$  for  $r = 4 \cdots 7$ . Substituting these values into Equation (8.32) gives the desired result. Figure 8.22 shows a graphical representation of the transform process with the original signal shown as the dotted line, the time-domain aliasing error as the dashed line, and the output signal of the transform process as the full line with dots showing the actual sample values.



**Figure 8.22** The output signal of the transform process is the sum of the original signal and the time-domain aliasing error. ■

Now using the transform process defined by Equations (8.28) and (8.29) a critically sampled overlapped transform with perfect reconstruction can be implemented. However, the properties of this transform process need to be combined with a suitable window function. This window function should have similar properties to the frequency response of perfect reconstruction bandpass filters, that is, when the weighting function is squared and added to a shifted version of itself, the resulting weighting is unity. An example of the coefficients of a possible weighting function for an eight-band TDAC transform is given in Table 8.1.

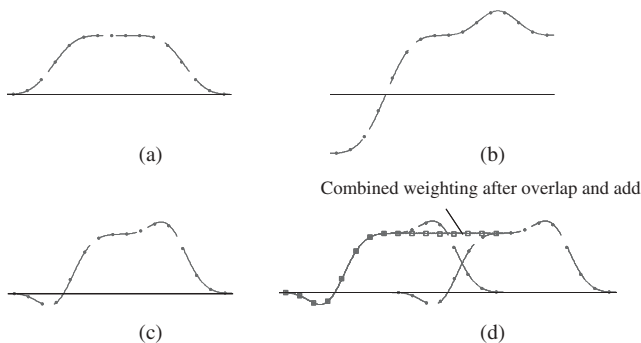
Figure 8.23 shows how the weighting function is combined with the overlapped transform process to provide perfect reconstruction of the original signal. Figure 8.23(a) shows the shape of the weighting function that is multiplied with the input

**Table 8.1** Weighting function coefficients for an eight-band TDAC transform.

| $n$ | $W(n)$  |
|-----|---------|
| 0   | 0.00443 |
| 1   | 0.06153 |
| 2   | 0.24757 |
| 3   | 0.55031 |
| 4   | 0.82831 |
| 5   | 0.96674 |
| 6   | 0.99788 |
| 7   | 0.99999 |

signal before the transform is applied. Then, if the forward and inverse transform is applied to the weighting function only, Figure 8.23(b) shows the distorted weighting function that is produced by time-domain aliasing in the transform process. This distorted weighting function is then multiplied by the original weights to produce the final weighting for the output of the transform process as shown in Figure 8.23(c). This final weighting function now exhibits the desired properties for time-domain aliasing cancellation. Figure 8.23(d) shows that, if the output of each transform process is overlapped by half with the previous output, the original input signal is reconstructed perfectly.

The frequency analysis and synthesis used in the Dolby AC-3 audio coding algorithm is based on the transform process defined by Equations (8.28) and (8.29). The transform is performed on a single block of 512 audio samples or two blocks of 256 samples, and consequently the window function used has a length of 512 but has a similar shape to the function shown in Figure 8.23.



**Figure 8.23** (a) The weighting function. (b) The distorted weighting function after the forward and inverse transform has been applied. (c) The final weighting function after multiplying the distorted function by the original weighting function. (d) If the final weighting function is overlapped by half and added, the result is a constant weighting of the reconstructed samples and a cancellation of the distortion introduced by the transform process.

## 8.7. SUMMARY

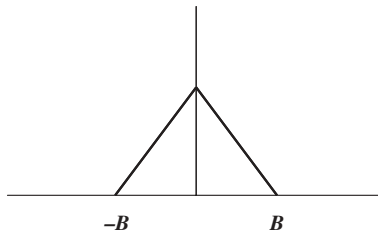
In this chapter, two different techniques for converting from time-domain samples to frequency-domain coefficients have been explained. In particular, the cosine-modulated filterbank approach used in the MPEG audio coding algorithms and the time-domain aliasing cancellation approach used in the Dolby AC-3 algorithm have been investigated. The following chapters explain how these frequency analysis and synthesis techniques have been incorporated into the design of the standard audio coding algorithms.

## PROBLEMS

8.1 Briefly explain what is meant by the following terms:

- (a) Sampling
- (b) Aliasing
- (c) Nyquist sampling rate

8.2 If a signal has the magnitude spectrum given below:



- (a) Sketch the spectrum of the sampled signal if the sampling frequency is  $2B$ .
  - (b) Sketch the spectrum of the signal if it has been subsampled by a factor of 2.
  - (c) Sketch the spectrum of the signal if it has been upsampled by a factor of 2.
- 8.3 Write the equations for the impulse response of the cosine-modulated analysis filters that would be used in a four-channel subband filterbank.
- 8.4 Write the equations for the impulse response of the cosine-modulated synthesis filters that would be used in a four-channel subband filterbank.
- 8.5 For the 32 PCM samples and the prototype filter impulse response given below, calculate the intermediate sum  $y(j)$  that would be produced in a two-channel analysis filterbank using Rothweiler's method.
- $$x(n) = \{-4.02, 4.76, 6.73, 7.60, 15.30, 9.17, -6.86, -7.36, -3.91, -7.68, -4.02, 4.76, 6.73, 7.60, 15.30, 9.17, -6.86, -7.36, -3.91, -7.68, -4.02, 4.76, 6.73, 7.60, 15.30, 9.17, -6.86, -7.36, -3.91, -7.68, -4.02, 4.76\}$$



Use these values to verify that the following equation holds for  $N = 4$  :

$$\sum_{k=0}^{N-1} \cos\left(\frac{2\pi k}{N} n\right) = \begin{cases} N, & \text{if } n = 0, N, 2N, \dots \\ 0, & \text{otherwise} \end{cases}$$

- 8.13** Determine the output signal obtained from the DCT described in Equations (8.28) and (8.29) when the input signal is  $x(n) = [0 \ 0 \ 4 \ 4 \ 4 \ 4 \ 0 \ 0]$ . Verify that this output signal is equal to the signal obtained using Equation (8.32).
- 8.14** Write the forward transform equation for a TDAC transform that would convert 16 input samples to eight coefficients.
- 8.15** Write the inverse transform equation for a TDAC transform that would convert eight coefficients into 16 output samples.
- 8.16** Calculate the output coefficients for the first two overlapping blocks of 16 samples from the following set of input sample values using the forward transform from question 16 and the following weighting function values.
- $w(n) = \{0.00443, 0.06153, 0.24757, 0.55031, 0.82831, 0.96674, 0.99788, 0.99999, 0.99999, 0.99788, 0.96674, 0.82831, 0.55031, 0.24757, 0.06153, 0.00443\}$
- 8.17** Calculate the output samples for the two blocks of eight coefficients from question 8.16 using the inverse transform from question 17 and the following weighting function values.
- 8.18** Add the last eight samples of the first block of output samples from question 19 with the first eight samples of the second block and verify that the resulting output samples are identical to the corresponding input sample values given in question 8.16.

## MATLAB EXERCISE 8.1

1. Modify the MATLAB code given in Section 8.5.1 to implement a 32-channel analysis filterbank. Use the MPEG-modified prototype filter given in Section 9.1.1 of Chapter 9 for the values of  $c(k)$ .
2. Construct a signal that is the sum of two sinusoids where the frequency of the first sinusoid is the center frequency of the first critical band of the ear and the frequency of the second sinusoid is 0.1 Bark higher than that of the first sinusoid.
3. Obtain the set of 32 subband samples that are produced by the analysis filterbank from part 1 (note that you should pass enough samples of the input signal through the filterbank so that the output subband samples are stable).
4. Repeat steps 2 and 3 keeping the frequency of the first sinusoid constant and increasing the frequency of the second sinusoid by 0.1 Bark until there is no overlap between the subband samples produced by the two sinusoids. Determine the difference in critical band rate required between the first and second sinusoids so that there is no overlap between the

subband samples produced by the two sinusoids and record this value in the following table:

| Critical band no. | Selectivity $\Delta z$ | Critical band no. | Selectivity $\Delta z$ |
|-------------------|------------------------|-------------------|------------------------|
| 1                 |                        | 14                |                        |
| 2                 |                        | 15                |                        |
| 3                 |                        | 16                |                        |
| 4                 |                        | 17                |                        |
| 5                 |                        | 18                |                        |
| 6                 |                        | 19                |                        |
| 7                 |                        | 20                |                        |
| 8                 |                        | 21                |                        |
| 9                 |                        | 22                |                        |
| 10                |                        | 23                |                        |
| 11                |                        | 24                |                        |
| 12                |                        | 25                |                        |
| 13                |                        |                   |                        |

- Repeat steps 2–4 for the remaining critical bands of the human auditory system.
- Comment on the selectivity of this filterbank compared with the selectivity of the human auditory system across the range of audible frequencies.

## MATLAB EXERCISE 8.2

- Use Equation (8.28) given in Section 8.6 to implement a 256 point TDAC forward transform. Use the Dolby AC-3 weighting function given in Section 10.1.3 of Chapter 10.
- Construct a signal that is the sum of two sinusoids where the frequency of the first sinusoid is the center frequency of the first critical band of the ear and the frequency of the second sinusoid is 0.1 Bark higher than that of the first sinusoid.
- Obtain the set of 256 coefficients that are produced by the forward TDAC transform from part 1 (Note that you should pass enough samples of the input signal through the transform so that the output coefficients are stable).
- Repeat steps 2 and 3 keeping the frequency of the first sinusoid constant and increasing the frequency of the second sinusoid by 0.1 Bark until there is no overlap between the subband samples produced by the two sinusoids. Determine the difference in critical band rate required between the first and second sinusoids so that there is no overlap between the

coefficients produced by the two sinusoids and record this value in the following table:

| Critical band no. | Selectivity $\Delta z$ | Critical band no. | Selectivity $\Delta z$ |
|-------------------|------------------------|-------------------|------------------------|
| 1                 |                        | 14                |                        |
| 2                 |                        | 15                |                        |
| 3                 |                        | 16                |                        |
| 4                 |                        | 17                |                        |
| 5                 |                        | 18                |                        |
| 6                 |                        | 19                |                        |
| 7                 |                        | 20                |                        |
| 8                 |                        | 21                |                        |
| 9                 |                        | 22                |                        |
| 10                |                        | 23                |                        |
| 11                |                        | 24                |                        |
| 12                |                        | 25                |                        |
| 13                |                        |                   |                        |

- 5. Repeat steps 2–4 for the remaining critical bands of the human auditory system.
- 6. Comment on the selectivity of this transform compared with the selectivity of the human auditory system across the range of audible frequencies.

REFERENCES

1. Rothweiler, J. H. Polyphase Quadrature Filters - A New Subband Coding Technique, *Proceedings of the IEEE International Conference on Acoustics Speech and Signal Processing*, 27.2, IEEE Press, Piscataway, N.J., 1983, pp. 1280–1283.

2. Princen, J., Bradley, A. Analysis/Synthesis filter bank design based on time domain aliasing cancellation, *IEEE Transactions on Acoustics, Speech and Signal Processing*, Vol. 34, Issue 5, Oct. 1986, pp. 1153–1161.